

follow a standard way of gathering data. They are all designed differently. However, it is important to know that most of the tools are now converging in terms of functionality (like tracing, profiling, etc), and lots of features are beginning to overlap with each other. Some well-known tools that provide tracing or trace-like functionality are:

- **SystemTap:** Uses a scripting technique somewhat similar to DTrace, with which you have to define your tracepoint in a specific script. Conditions could be associated with each tracepoint and written in a C-like SystemTap script. The script is then compiled to a kernel module before tracing starts. For dynamic tracing in the kernel, SystemTap uses *kprobes* and for static tracing, the *TRACE_EVENT* macro. In userspace, dynamic tracing is provided by the *uprobes* kernel module and Dyninst (pure user-space dynamic instrumentation).
- **Ftrace:** This is quite a popular tool among kernel developers and is part of the mainline kernel. Dynamic traces are based on *kprobes* and the static ones use the *TRACE_EVENT* macro. However, you can't define conditions, and only filtering of the traces can be done. This is quite fast compared to other tools but is mainly aimed at the kernel guys.
- **GDB:** Surprise! In case you didn't know, your friendly neighbourhood debugger doubles up as a tracer too. Userspace dynamic tracing can be used by the *trace* command, which is a TRAP-based technique to collect data. Use the *ftrace* command (not to be confused with Ftrace described above) for very fast tracing but remember that it can't be used everywhere in the code. Conditions for trace collection can be provided, which are bytecode translated and executed at runtime. You can also use *KGTP* for dynamic tracing, which makes the kernel expose a GDB remote interface onto which any GDB's instance can get hooked, and use the available trace commands.
- **Perf:** This is also a popular Swiss-army knife kind of analysis tool. Developers use this mainly to gather hardware performance counter data or gather information about bottlenecks in process execution. The tracing features of Perf are similar to Ftrace, though Ftrace is better in terms of performance.
- **Ktap:** This is a very recent entry into the tracing arena. It functions almost exactly like DTrace as seen from its bytecode-based dynamic tracing scripts, which are very easy to implement. A lightweight VM is implemented in the kernel itself for bytecode interpretation of tracepoint scripts. It is not yet in the mainline kernel though.

LTTng

Linux Trace Toolkit next generation is a very fast and extremely low-overhead kernel and userspace tracer. Low overhead, in simple terms, means that even with a 'non-activated' tracepoint inserted in the code, it gives near-

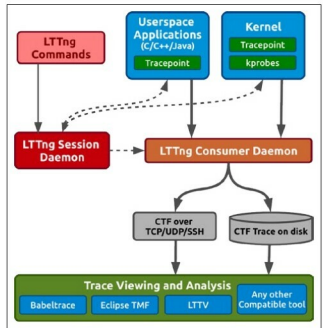


Figure 1: A typical tracing process flow with LTTng

zero impact on the overall execution of the target application. This makes LTTng a bit different from the other tools and a default choice for real time applications. Its tracing technique implements a fast wait-free read-copy-update (RCU) buffer for storing data from tracepoint execution.

In Figure 1, you can see that the LTTng session daemon acts as a focal point for trace control. An instrumented application, which contains the user's desired tracepoints, automatically registers itself to the session daemon just as its execution starts. So is the case with the kernel too. This is useful for handling simultaneous trace control for multiple trace sessions. From now on, the session daemon will manage all the tracing activity. The LTTng consumer daemon is responsible for handling the trace data coming from the applications. It exports raw trace data and builds a CTF to be written on the disk. The Common Trace Format (CTF) is a compact binary format, which stores all the trace data in a very well structured manner for further analysis by certain trace viewers and converters such as Babeltrace (command line), Linux Trace Toolkit Viewer (LTTV - graphical) or the Eclipse Tracing and Monitoring Framework (TMF - graphical). For example, you can view the exact time and the control flow through the various calls in the kernel, graphically, in a timeline using TMF, instead of wading through thousands of lines of trace data manually, as in Babeltrace's output. We'll cover more about using TMF efficiently later, when we discuss analysing traces graphically to diagnose bottlenecks in the system.

The joys of tracing

Well, going further, let's get our hands dirty with some tracing, and see all the insights a trace analysis can give about our kernel or application. The current stable release of LTTng is 2.3 Dominus Vobiscum.